Studio 1 — Data ER + Retirement MC · Tonight in 20 steps

> **The thesis tonight: when you have data, explore it; when you don't, simulate it.** Part 1 = 117,888 real PPP loans (data you have). Part 2 = your retirement in 35 years (data you don't). Everything runs in the browser — nothing to install.

Set up steps 1–4

1. **Hand in Math Check #1** (paper). Didn't finish? Upload your scan to HuskyCT by **Friday, Sept 11**.
2. **Sign in to github.com.** No `opim5641-work` repo yet? **+ New repository** → name it exactly that → Create. (Public or private — either is fine.)
3. Open HuskyCT → Module 1 → **Studio 1** page → click **Open the PPP Data ER notebook in Colab**.
4. Sign in with any Google account if Colab asks. You should see the notebook with empty `YOUR CODE HERE` cells — that's right, we fill them together.

Part 1 — Data ER: PPP loans in CT steps 5–12 · in pairs

5. **Run the import + load cell** (▶ or Shift+Enter). The 54 MB CSV loads from the course repo — give it ~20 seconds. `df.shape` should say **(117888, 53)**.
6. Keep the **data dictionary** open (linked in the notebook) — official meaning of every column, and which ones lie.
7. **Triage:** `df.info()`, then the missing-value census. With your partner: which holes actually matter?
8. **Duplicates:** check `LoanNumber` (unique) vs `BorrowerName` (repeats!). Why is a repeated business *not* necessarily a duplicate?
9. **Describe the money:** `describe()`, then the log-scale histogram. Find the top-10 loans — recognize anyone?
10. **Mean vs median** on `InitialApprovalAmount` — same muscle as Math Check #1. What does the gap say?
11. **Sectors + cities + dollars-per-job:** run the NAICS grouping, the city counts (spot the spelling dirt), and the `dollars_per_job` smell test.
12. **Write your 3 findings** at the bottom: each = one plot or table + two sentences. Then **File** → **Save a copy in GitHub** → `opim5641-work`. ★ This is the habit.

Part 2 — Retirement Monte Carlo steps 13–18

13. Back to the Studio 1 page → **Open the Retirement Monte Carlo notebook in Colab**.
14. **One future:** `np.random.seed(5641)`, one for-loop, one 35-year path. Run it twice — why is one run worthless?
15. **Ten thousand futures:** wrap the loop, histogram the endings, read the 25th / median / 75th lines.
16. **Spaghetti plot:** every future at once. Find the median path in the noodles.
17. **The twist:** replace `np.random.normal(0.07, 0.20)` with `sp['sp500_total_return'].sample(n=years, replace=True).values` — resample **98 years of real S&P history**. One line changes; the tails get fat.
18. **Answer with a probability:** P(≥ $1M) and P(< $500k) under both beliefs. Which one would you plan your life around?

Wrap steps 19–20

19. **File** → **Save a copy in GitHub** again → both notebooks now live in your repo. Second rep of the habit.
20. Check your repo on github.com — two notebooks there = you're done. Preview of Week 3: brute force (feel the explosion).

Don't panic

- **Colab disconnected?** Runtime → Run all. Nothing is lost that a re-run can't rebuild.
- **Data won't load?** It streams from a URL — check wifi, re-run the cell. Nothing to download by hand.
- **Missing ≠ zero.** A blank `ForgivenessAmount` is a question, not a $0.
- **Save a copy in GitHub** grayed out? You skipped the GitHub sign-in — Colab will prompt you.
- Behind? Fine. The **filled** versions of both notebooks are in the same repo folder — catch up any time.

Done at 7:30 when...

- Your `opim5641-work` repo holds **both notebooks**
- Your PPP notebook has **3 findings**, each with a plot/table + 2 sentences
- You've seen the normal vs bootstrap histograms disagree — and can say *why* in one sentence
- You answered a life question **with a probability**, not a single number
- Math Check #1 is out of your hands (paper tonight or upload by Friday)